

For example, if you want to make DOM measurements like getting the scroll position or other styles of an element, and then make mutations or trigger synchronous re-render, *useLayoutEffect* is our guy. If you don't need to do all this, you should use *useEffect*. I'll go as far as to say that 99 per cent of the time, you would be using *useEffect*.

Custom hooks

You may have heard a lot about custom hooks. Let us try and understand what they are exactly. Custom hooks are normal JavaScript functions that can use other hooks inside them, and contain a common stateful logic that can be reused within multiple components. These functions are prefixed with the word 'use'. You can name them anything as long as they have the 'use' prefix. Let's say you have two functions or components which implement some common logic. You can create a third function with this common logic, and implement it in the other two functions. After all, hooks are just functions. They give us the advantage of fewer keystrokes and less repetitive code. I will show how this happens and we'll create our very own hook now.

Let's say we have a *count* state variable and we want to store it in the local storage whenever the value updates. *useEffect* is the perfect candidate here.

```
function App() {
  const [count, setCount] = useState(() => {
    return JSON.parse(
      window.localStorage.getItem('counter-value') ||
      '0'
    );
  });
  useEffect (() => {
    window.localStorage.setItem('counter-value', count);
  }, [count]);

  return (
    <div>
      <button onClick={() => {
        setCount(count + 1)
      }}>{count}</button>
    </div>
  );
}
```

But what if we want to update this value in ten other components? The highlighted code will be repeated in all these 10 components. To avoid this repetition, we will use a custom hook.

```
function useLocalStorage(key, default) {
  const [state, setState] = useState(() => {
    return JSON.parse(
```

```
      window.localStorage.getItem(key) ||
      String(default)
    );
  });
  useEffect (() => {
    window.localStorage.setItem(key, state);
  }, [state]);
}

function App() {
  const [count, setCount] = useLocalStorage('counter-
value', 0);
  return (
    <div>
      <button onClick={() => {
        setCount(count + 1)
      }}>{count}</button>
    </div>
  );
}
```

As you see in the above code, we have created our very own custom hook called *useLocalStorage* and added the repeated code to this hook. Now, in all our 10 components we can simply call the *useLocalStorage* hook, and pass in the key and the default value. It's as simple as that and will work in the exact same way as before.

With the introduction of custom hooks, people started creating hooks for the most common use cases. A simple search for a local storage hook in npm gave 60 already existing packages for various use cases. So whenever you find the use case for a custom hook in your React application, I recommend going through this list; maybe someone has already created a package that you can reuse and save development time.

This is one of the strengths of having such a vast and active community and React has that -- you can surely use it to your advantage.

useDebugValue

```
function useLocalStorage (key, default) {
  const [count, setCount] = useState(0);

  // ...

  // Show a label in DevTools next to this Hook
  // e.g. "LocalStorage: Empty" useDebugValue (count == 0 ?
  'Empty': 'Not Empty');
  useDebugValue(count == 0 ? 'Empty' : 'Not Empty');

  return count;
}
```

Continued to page...70